机器臂关节角路径的多目标优化设计

摘　要

机器人手臂是一种由多个连接杆和关节组成的自动设备，广泛用于工业制造、精细操作、危险环境作业以及物流领域。本文通过建立多目标优化模型，对机器臂抓取特定位置货物的关节角路径进行优化，从而规划各个关节角的最优路径，实现抓取任务。

针对问题一，该问本文建立了 **基于浮点遗传算法的单目标优化模型。**首先使用标准 DH 参数法建立 **多重复合空间变换矩阵，**转换多自然坐标系为基准坐标系，求解正运动学模型得机器臂末端执行器位置，并用机器人工具箱进行仿真绘出简图，见本文图 4。发现该六自由度三连杆机器臂只有三个自由度会影响机器臂末端执行器位置，令为参数，用机器臂末端执行器位置与目标点欧式距离定义出优化目标函数，结合约束条件用 **浮点遗传算法** 求解得：$\theta_1 = 0.6209, \theta_2 = 0.0989, \theta_3 = 0.3428, \theta_4 = -4.8207, \theta_5 = 0, \theta_6 = 2.0971$

针对问题二，在优化过程中加入对能耗的考量后构建 **基于多元遗传算法的多元优化模型** ，得到一系列 Pareto 最优解。采用层次分析法得到三个指标的权值向量，对 Pareto 最优解集进行排序择优。**得到最优关节角路径为：**$\theta_1 = 0.6208, \theta_2 = 0.1001, \theta_3 = 0.3411, \theta_4 = -4.2826, \theta_5 = 0, \theta_6 = 3.0476$

针对问题三，在二维平面上采用栅格法对环境进行划分，在问题二的基础上建立基于 RRT **算法** 的最小路径规划模型，并用贪心算法优化路径。最优路径如本文图 14所示。

针对问题四，针对问题四，我们需要求解一个涉及多目标点的最优总路径规划。针对路径闭合以及要过所有目标点两个限制，我们建立哈密尔顿圆环以满足。再类同第三问求解出任意两点间的最短距离，运用分支限界算法结合优先队列（最小堆）来实现对最优路径的规划选取。最终总路径为 start-1-2-5-3-4-start，路径长度为 98 个单元格（19600mm）。

关键词：浮点数遗传算法 正运动学 多目标优化 RRT 哈密尔顿回路

1 问题重述

1.1 题目背景

机器臂由多个连杆和关节组成，广泛用于工业生产、精密操作、危险环境作业和物流领域。研究重点包括运动学与动力学建模、关节角路径优化设计和路径规划。

1.2 求解问题

(1) 给出机器臂初始参数，绘制其在零位状态（$\theta_1 = 0°$、$\theta_2 = -90°$、$\theta_3 = 0°$、$\theta_4 = 180°$、$\theta_5 = -90°$、$\theta_6 = 0°$）的六自由度机器臂的简图，建立运动数学模型，并以最小化末端误差为目标优化关节角路径。

(2) 在已知机器臂质量和末端载重、关节转动惯量和平均角速度的情况下，以最小化末端误差和能耗为目标优化关节角路径。

(3) 在存在障碍物的情况下，以最小化末端误差和能耗为目标设计最优底座移动路径和关节角路径。其中，底座移动路径用栅格图可视化。

(4) 在需要绕过障碍物抓取多个货物的情况下，以最小化末端误差和能耗为目标设计最优底座移动路径和关节角路径，并计算出此时总末端误差和总能耗。

2 问题分析

2.1 问题一的分析

问题一为以最小化末端误差为目标，六个优化关节角路径为优化参数的单目标优化问题。但由于关节 4、5、6 之间没有连杆，所以在求解机器臂末端执行器位置时关节 4、5、6 的关节角路径改变对最终结果并无影响，即此问题实为三参数的单目标优化问题，考虑使用浮点遗传算法求解。对于单目标优化问题，我们需找出目标函数和约束条件，此处约束条件较少且已由题给机器臂初始参数决定。目标函数可由机器臂末端执行器实际到达的位置与目标点的欧氏距离表示，求解机器臂末端执行器的位置我们可考虑用标准 D-H 参数法推导出正运动学模型，采用 matlab 机器人工具箱进行仿真。

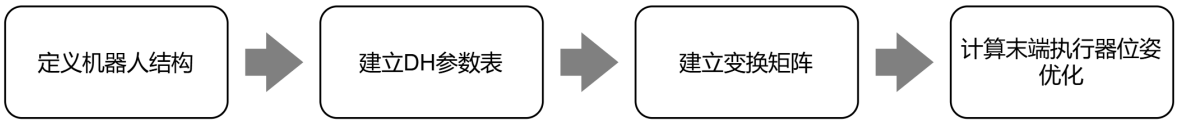


Figure 1: 正运动分析流程图

2.2 问题二的分析

问题二在问题一的基础上加入了对能耗的考量，使单目标优化问题转换为了多目标优化问题，我们可建立力学模型得到第二个优化目标函数。对于多目标优化问题，考虑先使用多目标的遗传算法，用 MATLAB 编程得 Pareto 最优解集，再对每个 Pareto 最优解进行加权，比较加权后的 Preto 最优解，求出最终最优解。运用层次分析法求得权重。

2.3 问题三的分析

问题三在问题二的基础上，规划最短路径控制机器臂底座移动绕过障碍物获取货物。在解决本问时可将其拆分为两个问题，首先为机器臂规划绕过障碍物的最短路径，接着以最小化末端误差和能耗为目标，设计最优关节角路径。后者我们已在问题二中详细分析。前者可考虑使用 RRT 随机生成树算法求解最短路径。

2.4 问题四的分析

问题四在问题三的基础上考虑了多个目标点的路径规划。也可拆分为两个子问题，首先为机器臂规划绕过障碍物抓到达多个目标点的最短路径，接着以最小化末端误差和能耗为目标，设计最优关节角路径。后者我们已在问题二中详细分析。前者由于算力限制，RRT 算法并不能返回可行解，我们考虑将问题转换为哈密尔顿回路解决。

3 模型假设

由于机器臂真实结构较为复杂，对机器臂系统进行如下处理：

1. 只使用转动副设计关节。（不考虑移动副的使用。）
2. 不考虑机器臂工作时关节摩擦产生的内能损失与电能产生的焦耳效应损耗。（即仅考虑调控关节转动以及克服机械臂重力做功消耗的能量）

3. 假设机器臂连杆质量在沿杆方向均匀分布。

4. 假设机器臂为刚体。

4 符号说明

表 1 列出了本文需要的符号，文中出现的其它符号将在出现时进行解释。

Table 1: 符号说明

序号	符号	符号描述
1	α_i	连杆扭角
2	d_i	连杆偏移量
3	a_i	连杆长度
4	θ_i	关节角度
5	P	机器臂的基座和末端之间坐标变换的过度矩阵
6	d_E	机器臂末端与目标点间的欧式距离
7	ω_i	关节 i 的转动角速度
8	t^i	关节 i 的转动时间
9	X	遗传算法基因
10	G_h	遗传算法染色体
11	E_p	重力势能
12	ΔE_R	转动能耗
13	M	机器臂系统和抓取物体的质量和
14	g	重力加速度
15	I_i	关节 i 的转动惯量
16	w	权重向量
17	K	RRT 算法最大采样数

5 模型建立与求解

5.1 问题一：单目标优化

5.1.1 模型的建立

对于优化问题首要找出目标函数和约束条件，该问题约束条件较少且已由题给机器臂初始参数决定。目标函数我们由机器臂末端执行器实际到达的位置与目标点的欧氏距离表示，目标点已知，只需求解机器臂末端执行器的位置。而正运动分析是机器人学中的一个重要研究方向，可用于计算机器人的末端执行器的位姿（位置和方向），本问中所需的机器臂末端执行器位姿即可通过建立正运动分析模型求解。

5.1.1.1 机器臂正运动学模型的建立

定义机器人结构 在正运动分析中，通常使用 Denavit-Hartenberg（DH）参数来建立机器人关节之间的关系，并通过连乘变换矩阵来计算末端执行器的位姿。其中，DH 法具体规则为：

1. 沿着关节 i+1 的方向选择轴 z_i；
2. 将原点 O_i 放置于轴 z_i 与轴 z_{i-1} 和轴 z_i 的公垂线的交点；
3. 将 $O_{i'}$ 放置于公垂线与轴 z_{i-1} 的交点；
4. 沿着轴 z_{i-1} 和 z_i 的公垂线选择轴 x_i，方向由关节 i 指向关节 i+1；
5. 最后，选择 y_i 构成右手系。

一般的机器臂由连杆与关节组成，连杆的运动学功能在于保持两端的关节轴线具有固定的几何关系。首先需确定机器人的连杆数和关节类型（旋转关节或平移关节）。由题给条件知，六个关节均采用旋转副，三根连杆分别连接关节 12、23、34。如图 3。

下面，根据 DH 法具体规则定义连杆坐标系（图 2）：其中，α_i 为轴 z_i 与轴 z_{i-1} 之间的夹角，d_i 为 $O_{i'}$ 在连杆坐标系 $i-1$ 中的 z 坐标，a_i 是连杆 i 的长度，θ_i 是轴 x_i 与轴 x_{i-1} 之间的夹角。

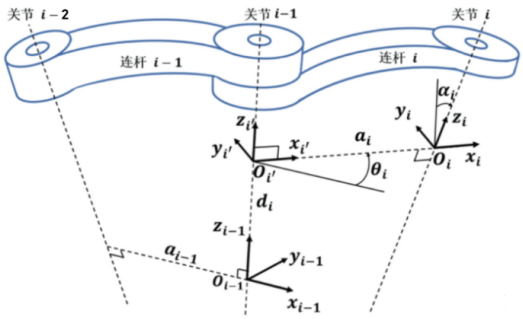


Figure 2: 标准 D-H 坐标系

连杆的自身特性决定于连杆长度 a_i，和连杆扭角 α_i。两连杆之间的关系通过它们的偏移量 d_i 和公共轴上的夹角 θ_i 定义。对于我们讨论的六自由度机器臂，a_i、α_i 和 d_i 是不变的，θ_i 是可变的。

建立 DH 参数表 题目中已经给出了 DH 参数表，如表 2。

Table 2: DH 参数表

关节 i	a_i/(mm)	$\alpha_i(°)$	d_i/(mm)	$\theta_i(°)$
1	0	0	600	-160~160
2	300	-90	0	-150~15
3	1200	0	0	-200~80
4	300	-90	1200	-180~180
5	0	-90	0	-120~120
6	0	-90	0	-180~180

为了进一步对机器臂进行运动学分析，我们建立零状态时的 DH 参数表（表 3，将其录入 matlab 机器人工具箱，绘制出零位状态六自由度机器臂简图，如图 3。

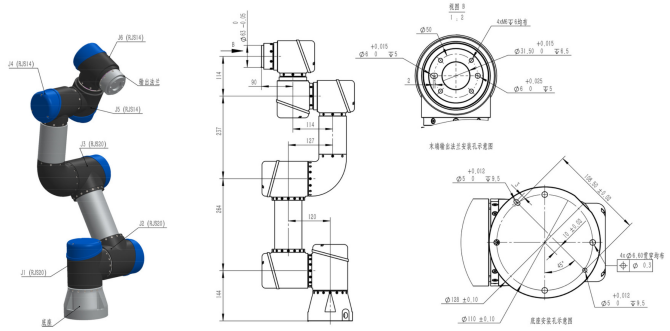


Figure 3: 六自由度机器臂结构

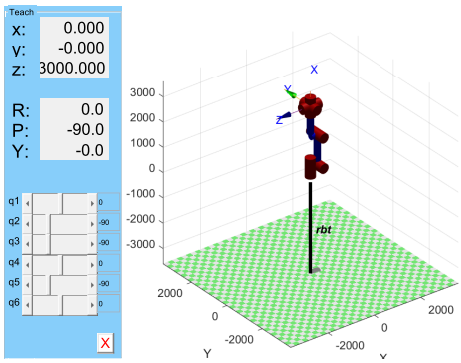
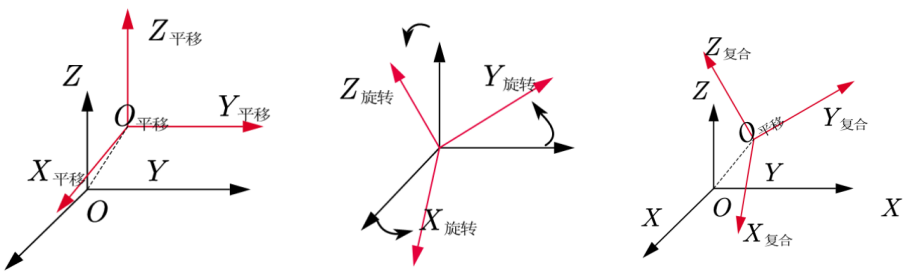


Figure 4: 零位状态六自由度机器臂简图

Table 3: 零状态 DH 参数表

关节 i	$a_i/(\mathrm{mm})$	$\alpha_i(°)$	$d_i/(\mathrm{mm})$	$\theta_i(°)$
1	0	0	600	0
2	300	-90	0	-90
3	1200	0	0	0
4	300	-90	1200	180
5	0	-90	0	-90
6	0	-90	0	0

建立变换矩阵 机器臂可被看作是一系列由关节连接起来的连杆，而 DH 标准坐标系为机器臂的每一连杆建立的坐标系。我们考虑建立从一个连杆坐标系 A 到另一个连杆坐标系 B 的空间坐标变换矩阵。讨论最一般的复合变换情况：坐标系 B 与 A 的原点既不重合，坐标轴方向也不相同。对于原点的改变，我们可以用平移变换实现；对于坐标轴方向的改变，我们可以用旋转变换实现。



(a) 坐标系平移变换　　(b) 坐标系旋转变换　　(c) 坐标系复合变换

Figure 5: 坐标系的旋转处理

a. **平移变换**：对于物体 p，易知坐标系平移可由坐标原点平移表示：

$$p_A = p_B + O_B^A \tag{1}$$

其中，p_A 为 p 在 A 下的坐标，p_B 为 p 在 B 下的坐标，O_B^A 为 B 坐标系原点到 A 坐标系原点的向量。

b. **旋转变换**：坐标系的旋转可由旋转矩阵（过渡矩阵）求得：

$$ {}_B^A R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & cos\theta & -sin\theta \\ 0 & sin\theta & cos\theta \end{bmatrix}, \ {}_B^A R_y = \begin{bmatrix} cos\theta & 0 & sin\theta \\ 0 & 1 & 0 \\ -sin\theta & 0 & cos\theta \end{bmatrix}, \ {}_B^A R_z = \begin{bmatrix} cos\theta & -sin\theta & 0 \\ sin\theta & cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{2}$$

c. **复合变换**：现在可讨论坐标系间的相对位置和姿态的复合变换：

$$p_A = {}_B^A R \cdot p_B + O_B^A \tag{3}$$

此时，变换式 3对于 p_B 而言是非齐次的，但是可以通过对矩阵增添列的方法将其表示成等价的齐次变换形式：

$$\begin{bmatrix} p_A \\ 1 \end{bmatrix} = \begin{bmatrix} {}_B^A R & O_B^A \\ 0 & 1 \end{bmatrix} \tag{4}$$

d. **复合齐次变换**: 下面我们再来讨论平移齐次坐标变换，还是讨论点 p 在 A 下的坐标变换到 B 下的坐标。$O_B^A = (a, b, c)^T$，则平移矩阵：

$$T = \begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 0 & c \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{5}$$

坐标系绕轴旋转角度依旧为 θ，则旋转矩阵：

$$R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos\theta & -sin\theta & 0 \\ 0 & sin\theta & cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, R_y = \begin{bmatrix} cos\theta & 0 & sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -sin\theta & 0 & cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, R_z = \begin{bmatrix} cos\theta & -sin\theta & 0 & 0 \\ sin\theta & cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{6}$$

那么复合齐次变换为：

$$^{A}_{B}P = T(a,b,c)R_xR_yR_z \tag{7}$$

计算末端执行器位姿 我们已经得到了由连杆坐标系 A 到连杆坐标系 B 的空间变换模型，现在将复合齐次变换式 7带入我们讨论的三连杆六自由度机器臂。

a. **位置** 在求解其末端执行器位置时可将其近似为三自由度机器臂（即只用考虑 $(\theta_1, \theta_2, \theta_3)$：

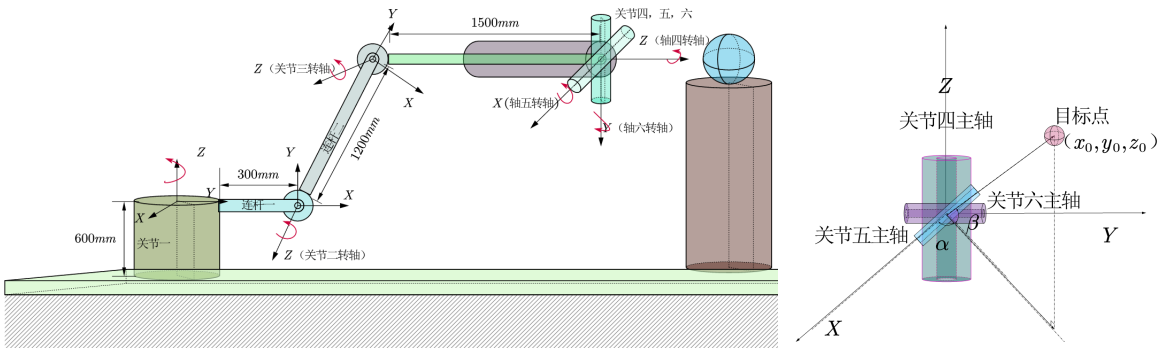
$$P = T_1(a_1,b_1,c_1)...T_3(a_3,b_3,c_3)R_{1x}R_{1y}R_{1z}...R_{3x}R_{3y}R_{3z} \tag{8}$$

最后将变换矩阵 P_3 带入坐标变换公式：

$$\begin{bmatrix} p_0 \\ 1 \end{bmatrix} = P \begin{bmatrix} p_3 \\ 1 \end{bmatrix} \tag{9}$$

即可得到机器臂末端执行器的坐标 p_3，为方便后续处理，我们将 p_3 记为 p。

b. **姿态（抓取朝向）**通过位置调整我们使机器臂末端尽可能靠近目标点，实现抓取目标还需调整机器臂末端执行器朝向。用机器人工具箱姿态调整进行仿真模拟，理想末端朝向 z_4 应和 $\overrightarrow{pp_{goal}}$ 相同。零位状态下，关节五与 x_4 重合，关节六与 y_4 重合，如图 6。



(a) 姿态调整示意图　　(b) 零位状态示意图

Figure 6: 末端执行器姿态调整

其中，α 为 $\boldsymbol{p}$ 在 x*o*y 平面上的投影与 x 轴所成角，β 为 $\boldsymbol{p}$ 与 x*o*y 面所成线面角。

$$\alpha = arccos\frac{x_p}{\sqrt{x_p^2+y_p^2}}$$
$$\beta = arctan\frac{z_p}{\sqrt{x_p^2+y_p^2}}$$

现使用式 6调整关节四、五、六使之朝向目标。首先调整 z_4（关节四）使 p 的投影与 x 轴重合，即关节四的关节角路径为 $\theta_4 = \alpha - 360°$；接着调整 y 轴（关节六）使 p 与 z 轴重合，即关节六的关节角路径为 $\theta_6 = 90° - \beta$。

优化模型的建立 至此，结合目标函数与约束条件的表达式，我们构建优化模型如下：

$$\min \quad d_E = (1500-x_p)^2 + (1200-y_p)^2 + (200-z_p)^2$$
$$s.t.\begin{cases} -160 \le \theta_1 \le 160, -60 \le \theta_1 \le 105, -200 \le \theta_1 \le 80 \\ \begin{bmatrix} p_0 \\ 1 \end{bmatrix} = \begin{bmatrix} {}^1_2R & O_2^1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} {}^2_3R & O_3^2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p \\ 1 \end{bmatrix} \\ p_0 = (a_3cos(90°+\theta), a_3sin(90°+\theta), 0)^T \\ p = (x_p, y_p, z_p)^T \\ O_2^1 = (a_2cos\theta_1, a_2sin\theta_1, 0)^T \\ O_3^2 = (a_3cos(\theta_2+90°), a_3sin(\theta_2+90°), 0)^T \\ {}^1_2R = \begin{bmatrix} cos\theta_1 & 0 & sin\theta_1 \\ sin\theta_1 & 0 & -cos\theta_1 \\ 0 & 1 & 0 \end{bmatrix}, {}^2_3R = \begin{bmatrix} cos\theta_2 & -sin\theta_2 & 0 \\ sin\theta_2 & cos\theta_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ a_2 = 300mm, a_3 = 1200mm \end{cases} \tag{10}$$

5.1.2 模型的求解

5.1.2.1 机器臂正运动学模型的求解

现求解出机器臂末端对地坐标：

$$\begin{bmatrix} p_3 \\ 1 \end{bmatrix} = \begin{bmatrix} cos\theta1 & 0 & sin\theta1 & a_2cos\theta_1 \\ sin\theta1 & 0 & -cos\theta1 & a_2sin\theta1 \\ 0 & 1 & 0 & n \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a_3cos(90°+\theta_2) \\ a_3sin(90°+\theta_2) \\ 0 \\ 1 \end{bmatrix}$$

可得 $p_3=(x_p,y_p,z_p)$。（由于 p_3 表达式过于复杂，此处不给出。）

5.1.2.2 优化模型的求解——浮点遗传算法

在这里我们首先考虑使用鲁棒性强而适用范围广的遗传算法求解优化模型。通过查阅文献我们发现，传统二进制编码对多极值函数的全局优化问题难以得到高精度解，并且在计算目标函数时需要把二进制码解码成十进制浮点数。而浮点数遗传算法很好地节省了算力，并且拥有更好的全局最优搜索能力。因此在本模型中我们考虑使用浮点数遗传算法求解。

初始化种群 用向量 $(\theta_1,\theta_2,\theta_3)$ 表示染色体 G，其中基因采用浮点数编码的方式，如 θ_l 为 $[-160,160]$ 之间的一个浮点数，它表示了关节 1 的关节角路径的变换范围。随机产生一组染色体 $G_h(h=1,2,...,n)$（其中 n 为一代种群中的个体数），G_h 各不相同，此为第一代种群。

适应度的定义 将我们的优化目标 $min\ d_E$ 改写为适应度：

$$f(x)=\begin{cases} c_{max}-d_E, & d_E<c_{max} \\ 0, & others \end{cases} \tag{11}$$

自然选择 将每代种群共 L 个染色体按适应值 $f(h)$ 由大到小排列 $(h=1,2,...,101)$，排在最前一位的个体性能最优，将它复制一个，直接进入下一代种群．下一代种群的另 L-1 个染色体则从前代种群的 n 个染色体中按概率 $P_{(h)}=q'(1-q)h-1(h=1,2,...,101)$，用轮转法选择个体 G_h，产生后代。其中 $q'=\frac{q}{(1-(1-q)n)}$,q = 0．08，种群代数增 1。

交叉与变异 对于交叉操作，我们基于浮点遗传算法，选择均匀算术交叉算子，设定交叉概率 $P_c=0.2$，对群体中两两随机配对的 G_i 和 G_j 的基因 $X_{i(t)},X_{j(t)}$ 按下式进行算术交叉产生两个新个体：

$$\begin{cases} X_c=\alpha X_{i(t)}+(1-\alpha)X_{j(t)} \\ X_d=\alpha X_{j(t)}+(1-\alpha)X_{i(t)} \end{cases} \tag{12}$$

变异操作则直接将染色体中某一选定的浮点数利用随机数代替即可。对于变异操作，我们在每代种群中以变异率 $P_{(m)}=0.2$ 对染色体进行变异。基于浮点遗传算法，我们将染色体中某一选定的浮点数利用随机数代替产生新个体。

对交叉和变异成功所获染色体求其适应值，并与其父染色体比较，择性能优者进入种群。

判断停止进化条件 我们以进化 100 代为终止条件停止搜索。此时的适应度为 9.81，且其值在后续迭代中保持稳定，此时的染色体为最优染色体：(0.62089741,0.0989395,0.34284826)。即关节一的最优关节角路径 θ_1 为 0.62089741，关节二的最优关节角路径 θ_2 为 0.0989395，关节三的最优关节角路径 θ_3 为 0.34284826。将机器臂末端执行器坐标 (x_p, y_p, z_p) 代入 5.1.1.1，求得 α、β。使用公式：

$$\begin{aligned} \theta_4 &= \alpha - 2\pi \\ \theta_6 &= \frac{\pi}{4} - \beta \end{aligned} \tag{13}$$

求解得 $\theta_4 = -4.820656767641063, \theta_5 = 0, \theta_6 = 2.097074736955972$（弧度制）。

即关节一到关节六的最优关节角路径：$\theta_1 = 0.6209, \theta_2 = 0.0989, \theta_3 = 0.3428, \theta_4 = -4.8207, \theta_5 = 0, \theta_6 = 2.0971$。

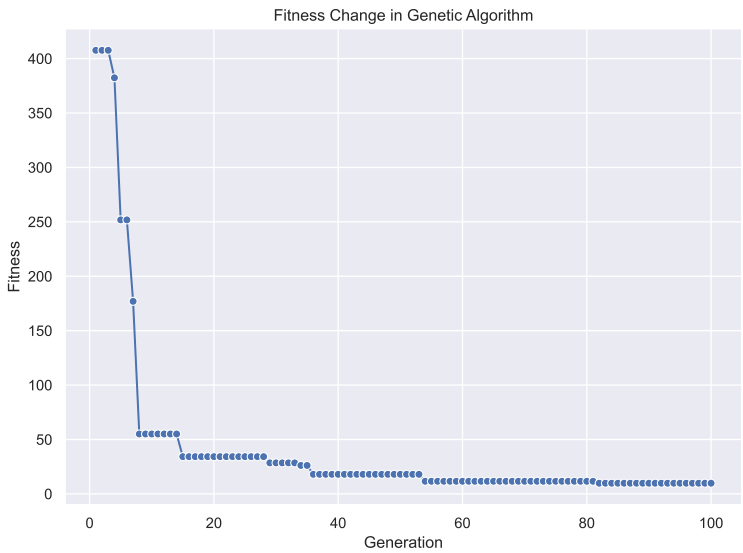


Figure 7: 适应度随进化代数的变化

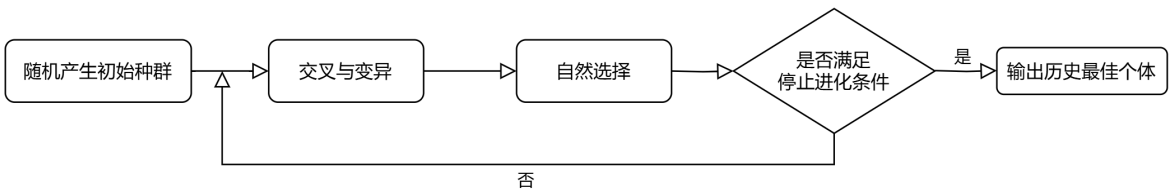


Figure 8: 浮点遗传算法流程图

5.2 问题二：多目标优化

5.2.1 模型的建立

本问在上文讨论的优化模型基础上增加了能耗的约束条件，我们先建立力学模型分析求解能耗的目标函数，接着建立基于多元遗传算法的多元目标优化问题。求得 Pareto 最优解集后考虑使用层次分析法确定最终最优解。

5.2.1.1 能耗模型的建立

转动能耗的确定 关节转动需要电机提供动能，动能与关节的转动惯量和速度相关，六个关节均可转动，此处优化参数即有六个：对于关节 i，其转动初始角度为 θ_i^0，转动末角度为 θ_i^t，转动角度为 θ_i，转动总时长为：

$$t_{total}^i = \frac{\theta_i}{\overline{\omega_i}}$$

转动角度间的关系为：

$$\begin{cases} \theta_i^t = \theta_i^0 + \theta_i \\ \theta_i^0 = \theta_i^0 \end{cases} \tag{14}$$

现已知 $(\theta_i^t, t_{total}^i)$、$(\theta_i^0, 0)$、$\omega_i^t = \theta_i^t(t)'$、$\omega_i^0 = \theta_i^0(0)'$ 四个条件，理论上可求解出式 15：

$$\theta_i(t) = b_0^i + b_1^i t^i + b_2^i (t^i)^2 + b_3^i (t^i)^3,\ t^i \in (0, t_{total}^i),\ i = 1, 2, .., 6 \tag{15}$$

对于关节 i，转动时间为 t 时，有刚体转动动能定理：

$$\Delta E_{Ri} = \frac{1}{2} I_i \omega_i(t)^2$$

对时间 t 积分：

$$\int_0^{t_{total}^i} \frac{1}{2} I_i \omega_i(t)^2 dt$$

除以 t 调整量纲：

$$\frac{\int_0^{t_{total}^i} \frac{1}{2} I_i \omega_i(t)^2 dt}{t_{total}^i}$$

最后，整个机器臂的转动能耗由六个关节处的转动能耗累和求得：

$$\Delta E_R = \sum_{i=1}^{6} \frac{\int_0^{t_{total}^i} \frac{1}{2} I_i \omega_i(t)^2 dt}{t_{total}^i} \tag{16}$$

克服重力做功能耗 机器臂连杆的抬升需要克服重力做功，机器臂系统的重力势能增大。现规定零势能面为关节一水平面，且假设连杆为质量随长度分布均匀的直杆，不

考虑关节质量。此时连杆的重心在其几何中心，则零位状态机器臂系统的重力势能为（如图 9）：

$$E_p^{origin} = Mg \cdot \frac{1}{2}(d_1 + a_2 + a_3 + a_4 + d_4) \tag{17}$$

如图 9，机器臂经过位姿调整抓取目标物的末状态系统重力势能为：

$$E_p^t = \frac{a_2 M g z_2}{d_1 + a_2 + a_3 + (a_4 + d_4)} + \frac{a_3 M g \cdot \frac{z_2+z_3}{2}}{d_1 + a_2 + a_3 + (a_4 + d_4)} + \frac{(a_4 + d_4) M g \cdot \frac{z_3+z_4}{2}}{d_1 + a_2 + a_3 + (a_4 + d_4)} \tag{18}$$

将 E_p^{origin} 与 E_p^t 做差即可得到克服重力做功导致的能耗。

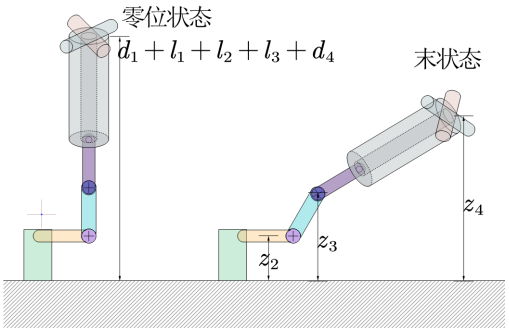


Figure 9: 重力势能求解示意图

总能耗 机器臂系统的总能耗 ΔE_p 即为其转动能耗与克服重力做功能耗之和：

$$\Delta E_p = E_p^{origin} - E_p^t + \Delta E_R \tag{19}$$

再将式 16、17、18带入，求得总能耗表达式。

5.2.1.2 优化目标归一化处理

此次需要优化的目标有 2 个，为多目标优化问题，而 2 个目标函数单位不同，故需要进行归一化处理统一量纲。归一化处理公式：

$$f^* = \frac{f - min\ f}{max\ f - min\ f} \tag{20}$$

5.2.1.3 多目标优化模型的建立

目标函数的确立 在 5.2.1.1 中已经给出了总能耗表达式，将其带入归一化处理公式（式 20）；再将问题一中末端误差表达式带入归一化处理公式（式 20），写出优化函数如下：

$$\begin{cases} d_E^* = \frac{d_E - min\ d_E}{max\ d_E - min\ d_E} \\ E^* = \frac{\Delta E_p - min\ \Delta E_p}{max\ \Delta E_p - min\ \Delta E_p} \end{cases} \tag{21}$$

约束条件的确立 结合 5.2.1.1 所建力学模型和 5.1.1.2 式 22，可求得约束条件 s.t.。值得注意的是，优化参数理论上有六个，θ_4、θ_5、θ_6 参与调控转动能耗。可经计算发现，转动能耗为常数，则优化变量同问题一中一样，只有 θ_1、θ_2、θ_3。

优化模型的建立 至此，结合目标函数与约束条件的表达式，我们构建优化模型如下：

$$
\begin{aligned}
\min \quad & d_E^* = \frac{d_E - min\ d_E}{max\ d_E - min\ d_E} \\
& E^* = \frac{\Delta E_p - min\ \Delta E_p}{max\ \Delta E_p - min\ \Delta E_p}
\end{aligned}
$$

$$
s.t. \begin{cases}
-160 \le \theta_1 \le 160, -60 \le \theta_1 \le 105, -200 \le \theta_1 \le 80 \\
\begin{bmatrix} p_0 \\ 1 \end{bmatrix} = \begin{bmatrix} {}^1_2R & O_2^1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} {}^2_3R & O_3^2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p \\ 1 \end{bmatrix} \\
p_0 = (a_3cos(90^\circ + \theta), a_3sin(90^\circ + \theta), 0)^T \\
p = (x_p, y_p, z_p)^T \\
O_2^1 = (a_2cos\theta_1, a_2sin\theta_1, 0)^T \\
O_3^2 = (a_3cos(\theta_2 + 90^\circ), a_3sin(\theta_2 + 90^\circ), 0)^T \\
{}^1_2R = \begin{bmatrix} cos\theta_1 & 0 & sin\theta_1 \\ sin\theta_1 & 0 & -cos\theta_1 \\ 0 & 1 & 0 \end{bmatrix},\ {}^2_3R = \begin{bmatrix} cos\theta_2 & -sin\theta_2 & 0 \\ sin\theta_2 & cos\theta_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\
\Delta E_p = E_p^{origin} - E_p^t + \Delta E_R \\
E_p^t = \frac{a_2Mgz_2}{d_1+a_2+a_3+(a_4+d_4)} + \frac{a_3Mg\cdot\frac{z_2+z_3}{2}}{d_1+a_2+a_3+(a_4+d_4)} + \frac{(a_4+d_4)Mg\cdot\frac{z_3+z_4}{2}}{d_1+a_2+a_3+(a_4+d_4)} \\
E_p^{origin} = Mg \cdot \frac{1}{2}(d_1 + a_2 + a_3 + a_4 + d_4) \\
d_1 = 600,\ a_2 = 300,\ a_3 = 1200,\ a_4 = 300,\ d_4 = 1200 \\
p_1 = (0,0,0)^T,\ p_2 = (a_2cos\theta1, a_2sin\theta1, 600)^T,\ p_3 = (x_3, y_3, z_3)^T \\
\begin{bmatrix} p_3 \\ 1 \end{bmatrix} = \begin{bmatrix} cos\theta_1 & 0 & sin\theta_1 & a_2cos\theta_1 \\ sin\theta_1 & 0 & -cos\theta_1 & a_2sin\theta_1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a_3cos(90^\circ + \theta_2) \\ a_3sin(90^\circ + \theta_2) \\ 0 \\ 0 \end{bmatrix} \\
M = 5kg,\ g = 9.8kg/s^2 \\
\Delta E_R = \sum_{i=1}^{6} \frac{\int_0^{t^i_{total}} \frac{1}{2} I_i \omega_i(t)^2 dt}{t^i_{total}} \\
\theta_i^t = \theta_i^0 + \theta_i,\ \theta_i^0 = \theta_i^0,\ t^i_{total} = \frac{\theta_i}{\overline{\omega_i}} \\
\theta_i(t) = b_0^i + b_1^i t^i + b_2^i (t^i)^2 + b_3^i (t^i)^3,\ t^i \in (0, t^i_{total}),\ i = 1, 2, .., 6
\end{cases}
\tag{22}
$$

5.2.1.4 层次分析法定权数

通过所建优化模型，我们已求解出其 Pareto 最优解集，现在对解集中的元素分别进行层次分析定权数，加权并解得最后最优解。考虑在准则层中将能耗分为克服重力做功能耗与转动能耗两个子目标，使最终决策更可靠。

层次结构模型 对机器臂进行设计时，需要根据末端误差的特性，对其赋予较大权重，提高它在总目标函数中的贡献度。对于机器臂而言，抓东西的功能比省电更为重要。因此构造层次分析模型（图 10），建立比较矩阵，计算出各个指标在最终评价中的权重系数。比较矩阵 Y 为

$$Y = \begin{bmatrix} 1 & Y_{12} & \cdots & Y_{1m} \\ \frac{1}{Y_{12}} & 1 & \cdots & Y_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{Y_{1m}} & \frac{1}{Y_{2m}} & \cdots & 1 \end{bmatrix} \tag{23}$$

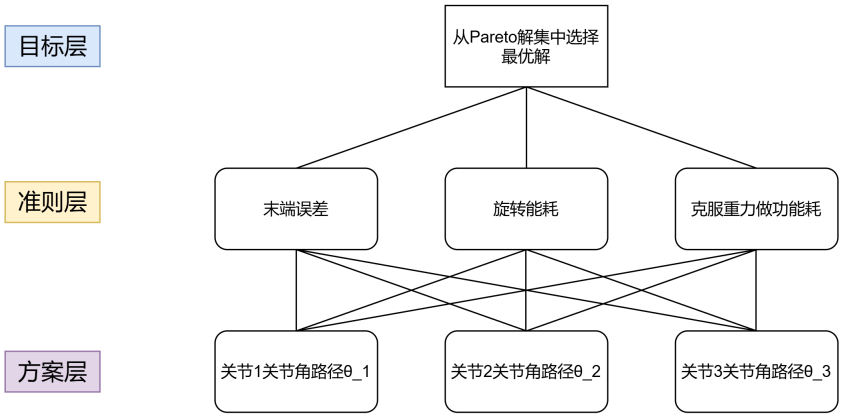


Figure 10: 层次结构模型

上述比较矩阵中的各比较标度与其含义如表 1 所示。

Table 4: 判断矩阵元素 a_{ij} 的标度方法

标度	含义
1	表示两个因素相比，具有同样重要性
3	表示两个因素相比，一个因素比另一个因素稍微重要
5	表示两个因素相比，一个因素比另一个因素明显重要
7	表示两个因素相比，一个因素比另一个因素强烈重要
9	表示两个因素相比，一个因素比另一个因素极端重要
2，4，6，8	上述两相邻判断的中值
倒数	因素 i 与 j 比较的判断 a_{ij}，则因素 j 与 i 比较的判断 $a_{ji} = 1/a_{ij}$

为衡量每一目标函数的重要性和影响，需要计算各目标函数的权重。对式 23中的各元素取算术平均值，以 w_i 表示权重，则算术平均权重系数的计算公式为

$$w_c = \frac{1}{m}\sum_{j=1}^{m}\sum_{k=1}^{m}\frac{Y_{ij}}{Y_{kj}} \tag{24}$$

同样的平均权重系数还有：几何平均权重系数 w_g，特征值法得平均权重系数 w_λ。

5.2.2 模型的求解

5.2.2.1 能耗模型的求解

现将 $(\theta_i^t, t_{total}^i)$、$(\theta_i^0, 0)$、$\omega_i^t = \theta_i^t(t)'$、$\omega_i^0 = \theta_i^0(0)'$ 带入式 15，可构造出方程组：

$$\begin{bmatrix}\theta_i^0\\ \theta_i^t\\ 0\\ 0\end{bmatrix} = \begin{bmatrix}1 & 0 & 0 & 0\\ 1 & t_i^{total} & (t_i^{total})^2 & (t_i^{total})^3\\ 0 & 1 & 0 & 0\\ 0 & 1 & 2t_i^{total} & 3(t_i^{total})^2\end{bmatrix}\begin{bmatrix}b_0^i\\ b_1^i\\ b_2^i\\ b_3^i\end{bmatrix} \tag{25}$$

通过上述方程组求得 $b_0^i, b_1^i, b_2^i, b_3^i$，将它们带入式 15，得到每个关节转动角度与转动时间的关系。

5.2.2.2 优化模型的求解——多目标遗传算法

多目标遗传算法大致思路与单目标浮点遗传算法类似，本文在 5.1.2.2 中已经详细讨论过，在此处不过多赘述。值得注意的是多目标遗传算法与单目标遗传算法的一些区别。首先，在多目标遗传算法中，我们不能得到唯一的最优解，算法返回值为所有互不

支配且不被其他解支配的 Pareto 解构成的解集。支配关系的数学描述为：设 p 和 q 是进化群体 Pop 中的任意两个不同的个体，称 p 支配 (dominate)q，则必须满足下列两个条件:

1. 对所有的子目标，p 不比 q 差，即 $f(p) \leq f(g), (k = 1, 2, ..., r)$

2. 至少存在一个子目标，使 p 比 q 好。即 $l \in 1, 2, ..., r$，使 $f(p) < f(g)$。

其中 r 为子目标的数量。在编程中我们很难实现如上描述，尝试改写上述条件。

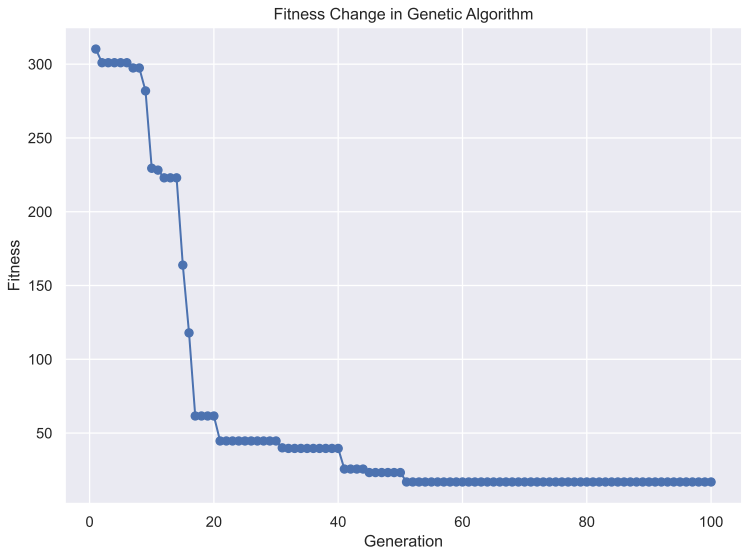


Figure 11: 适应度随进化代数的变化

5.2.2.3 层次分析比较求解最优关节角路径

填写比较矩阵：

$$\begin{bmatrix} 1 & 5 & 7 \\ \frac{1}{5} & 1 & 5 \\ \frac{1}{7} & \frac{1}{5} & 1 \end{bmatrix} \tag{26}$$

解得算术平均法权数向量：$w_c = (0.70865015, 0.22286756, 8.06848228)$，

几何平均法权数向量：$w_g = (0.71470956, 0.21849437, 0.86679607)$，

特征值法归一化处理后：$w_\lambda = (0.71470956, 0.21849437, 0.06679607)$，

综合三者得最终权重为 $w = (0.708650150.222867568.06848228)$。

将权值向量带入 Pareto 解集，筛选得最优解：$\theta_1 = 0.6208, \theta_2 = 0.1001, \theta_3 = 0.3411$。再将机器臂末端执行器坐标 (x_p, y_p, z_p) 代入 5.1.1.1，求得 α、β。使用公式 13，求得 $\theta_4 = -4.2826, \theta_5 = 0, \theta_6 = 3.0476$。

即关节一到关节六的最优关节角路径：$\theta_1 = 0.6208, \theta_2 = 0.1001, \theta_3 = 0.3411, \theta_4 =$

$-4.2826, \theta_5 = 0, \theta_6 = 3.0476$

5.2.3 结果分析

5.2.3.1 一致性检验

一致性指标 CI 求解公式为：

$$CI = \frac{\lambda_{max} - n}{n - 1}$$

通过查阅表格我们知道当 n=3 时 RI=0.52,计算出最大特征值 $\lambda_{max} = 3.1827668057021166$，带入一致性检验比率 CR 公式：

$$CR = \frac{CI}{RI} = 0.09138340285105828 < 0.1$$

符合一致性检验。

5.3 问题三：最短路径规划

5.3.1 模型的建立与求解

在问题二的基础上，规划最短路径控制机器臂底座移动绕过障碍物获取货物。在解决本问时可将其拆分为两个问题，首先为机器臂规划绕过障碍物的最短路径，接着以最小化末端误差和能耗为目标，设计最优关节角路径。后者我们已在问题二中详细分析，由于时间问题，在此不过多赘述。前者考虑使用快速扩展随机生成树算法（RRT）求解最短路径。

采用栅格法对环境进行划分，在栅格图上标出障碍物位置（如图 12）。现将机器人的工作空间视为二维平面，机器人视为质点，质点在 20*20 有限区域从起始点 S 移动到目标点 T。路径规划的优化准则为路径最短即寻找一条从 S 到 T 避开障碍物的最短路径。机器人在一个栅格上可以沿上、下、左、右这 4 个方向到达相邻的栅格.

Figure 12: 障碍物位置

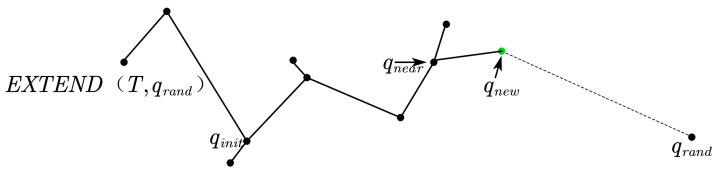


Figure 13: RRT 算法示意

在 RRT 中顶点坐标是连续的，在环境范围内生成顶点坐标随机值，接着算法将构建具有连续坐标的顶点的树。为了确定某个顶点或边是否碰到了障碍物，将对连续坐标进行取整，并在包含环境映射的 Matlab 变量中检查取整坐标。

随机采样 构建从起始位置到目标位置的树，并生成连接两个顶点的路径。由于障碍物的存在，树型不能直接朝着目标点延伸。采用随机采样方法：采样空间中随机一个点，生成树以概率 P_g 向目标点延伸，概率 P_s 向采样点延伸。延伸距离为 Δq. 对延伸路径进行碰撞检测，若没有与障碍物发生碰撞，则将这个新点也加入 RRT 树。

终止条件 RRT 生成树生成到目标点终止，若之前没能找到目标，最大采样数为 K。使用 K=10000,Δq=20,P_s=0.3，得到如图??所示的结果。蓝色的树从 qstart 开始，红色的是直到 qgoal 的路径。

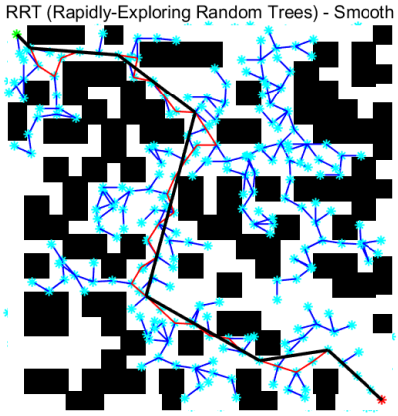


Figure 14: 最短路径

平滑——贪心算法 接下来编写平滑算法获得更短噪声更少的路径。在这里我们使用贪心算法从 q_start 连接 q_goal，如果失败，从更近的位置尝试直到连接。一旦 g_goal 连接成功，从它直接连接的位置开始连接下一段路径。平滑后的路径如图 14。

5.4 问题三：多目标点路径规划

5.4.1 模型的建立

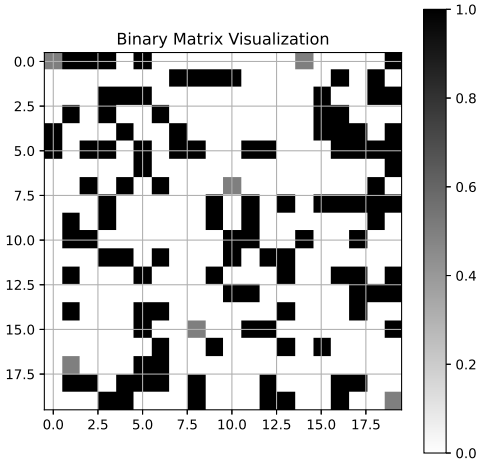


Figure 15: 障碍物与目标点位置

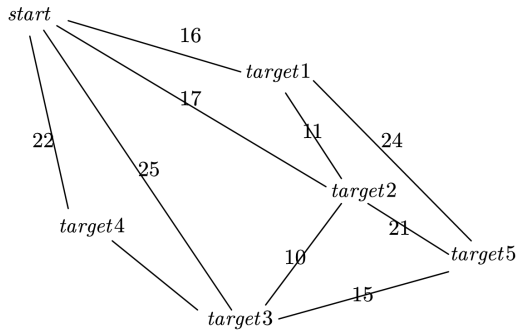


Figure 16: RRT 算法示意

目的：找到一次完整的移动抓取路径（回到原点的封闭路径），并且使移动总路径最小，故建立如下优化目标：

$$\min \quad \sum_{i=1}^{4} L_{path}$$

(27)

结合约束条件：

1. 由随机树算法生成任意两个节点之间的最短移动距离，构建最短距离无向赋权矩阵。
2. 构建节点间的邻接关系矩阵。

5.4.2 模型的求解

由于待求解的模型符合两个性质：

1. 要回到原点的封闭路径

2. 要过所有点

故我们考虑运用哈密尔顿圆的性质来完美贴合上述两种限制。现在将问题就转化为了在已有任意两点间最短距离的赋权连接图中，找一个权值加和最小的哈密尔顿圆。

在处理最小哈密尔顿圆我们运用破圈法处理：分支限界算法结合优先队列（最小堆）。

得到最终结果为：

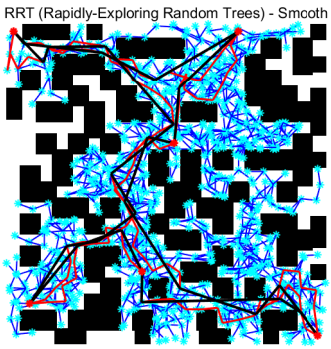


Figure 17: 最短路径

最短路径为：98*200mm=19600mm，路径为 0-1-2-5-3-4-0

6 附录

第一问代码 import matplotlib.pyplot as plt import numpy as np from math import cos, sin, pi, acos, atan, sqrt import seaborn as sns sns.set() Constants and parameters POP$_SIZE = 200NUM_PARAMS = 3NUM_GENERATIONS = 100MUTATION_RATE = 0.1CROSSOVER_PROB = 0.8$

Initialize population def initialize$_population(pop_size, num_params) : returnnp.random.rand(pop_size$

Compute end-effector position def endpoint(x): a1 = (320*x[0] - 160) * pi / 180 a2 = (165*x[1] - 60) * pi / 180 a3 = (280*x[2] - 200) * pi / 180

l1 = 300 R2$_1 = np.array([[cos(a1), 0, sin(a1), l1 * cos(a1)], [sin(a1), 0, -cos(a1), l1 * sin(a1)], [0, 1, 0, 600], [0, 0, 0, 1]])$

l2 = 1200 R3$_2 = np.array([[cos(a2), -sin(a2), 0, l2*cos(a2+pi/2)], [sin(a2), cos(a2), 0, l2* sin(a2 + pi/2)], [0, 0, 1, 0], [0, 0, 0, 1]])$

l3$_4 = 1500X = np.array([l3_4 * cos(pi/2 + a3), l3_4 * sin(pi/2 + a3), 0, 1])$

R = R2$_1@R3_2y = np.dot(R, X)$

return y

Fitness function (RMSE) def fitness(x): P = endpoint(x) target = np.array([1500, 1200, 200]) return np.sqrt(np.sum((P[:3] - target)**2))

Tournament selection def tournament$_selection(population, fitness_scores, tournament_size = 3) : selected_parents = []pop_size = len(population)$

for $_inrange(pop_size) : tournament_indices = np.random.choice(pop_size, size = tournament_size, re$ $False)tournament_fitness = fitness_scores[tournament_indices]selected_parents.append(population[tou$

return np.array(selected$_parents)$

Single-point crossover def crossover(parents, crossover$_prob = CROSSOVER_PROB) :$ $ifnp.random.rand() < crossover_prob : crossover_point = np.random.randint(1, len(parents[0]))child1$ $np.concatenate([parents[0][: crossover_point], parents[1][crossover_point :]])child2 = np.concatenate([pa$ $crossover_point], parents[0][crossover_point :]])else : child1, child2 = parents[0].copy(), parents[1].copy()$

return child1, child2

Mutation def mutate(child, mutation$_rate = MUTATION_RATE) : foriinrange(len(child)) :$ $ifnp.random.rand() < mutation_rate : child[i] = np.random.rand()returnchild$

Genetic algorithm def genetic$_algorithm(num_generations, pop_size, num_params) : population =$ $initialize_population(pop_size, num_params)best_fitness_history = []$

for generation in range(num$_g enerations) : fitness_s cores = np.array([fitness(x) for x in population$
$np.min(fitness_s cores) best_i ndividual = population[np.argmin(fitness_s cores)] best_f itness_h istory.appe$

next$_p opulation = [best_i ndividual] Elitism : preservethebestindividual$

while len(next$_p opulation) < pop_s ize : parents = tournament_s election(population, fitness_s cores) c$
$crossover(parents) child1 = mutate(child1) child2 = mutate(child2) next_p opulation.extend([child1, chi$

population = np.array(next$_p opulation[: pop_s ize])$

Final best individual and fitness final$_f itness_s cores = np.array([fitness(x) for x in population]) best_i$
$population[np.argmin(final_f itness_s cores)] best_f itness = np.min(final_f itness_s cores)$

创建图像 plt.figure(figsize=(8, 6))

使用 Seaborn 的线性回归图绘制数据 sns.lineplot(x=range(1, len(best$_f itness_h istory)+$
$1), y = best_f itness_h istory, marker =' o', color =' b')$

设置标签和标题 plt.xlabel('Generation') plt.ylabel('Fitness') plt.title('Fitness Change in Genetic Algorithm')

显示网格 plt.grid(True)

调整布局 plt.tight$_l ayout()$

显示图像 plt.show()

return best$_i ndividual, best_f itness, best_f itness_h istory return best_i ndividual, best_f itness, best_f itness$

Run the genetic algorithm best$_i ndividual, best_f itness, best_f itness_h istory = genetic_a lgorithm(NU$
$", best_i ndividual) print("\quad : ", best_f itness)$

"' 求解后三个调控参数"' 输出目标点在坐标系四中的坐标, 为了求解关节 4,5,6 转角 def spacechange(x): a1=(320*x[0]-160)*pi/180 a2=(165*x[1]-60)*pi/180 a3=(280*x[2]-200)*pi/180 l1=300 R2$_1 = np.array([[cos(a1), 0, sin(a1), l1 * cos(a1)], [sin(a1), 0, -cos(a1), l1 *$
$sin(a1)], [0, 1, 0, 600], [0, 0, 0, 1]]) l2 = 1200 R3_2 = np.array([[cos(a2), -sin(a2), 0, l2 * cos(a2+$
$pi/2)], [sin(a2), cos(a2), 0, l2 * sin(a2+pi/2)], [0, 0, 1, 0], [0, 0, 0, 1]]) l3_4 = 1500 R4_3 = np.array([[0, cos(a3)$
$a3), l3_4 * cos(pi/2+a3)], [0, sin(a3), sin(pi/2+a3), sin(pi/2+a3) * l3_4], [1, 0, 0, 0], [0, 0, 0, 1]]) X =$
$np.array([1500, 1200, 200, 1]) R = R2_1 @ R3_2 @ R4_3 \quad A \quad R_i nv = np.linalg.inv(R) y = np.dot(R_i nv, X) ret$
$P = spacechange(x) x0 = P[0] y0 = P[1] z0 = P[2] cos = x0/sqrt(x0 * *2 + y0 * *2) a =$
$acos(cos) tan = z0/sqrt(x0 * *2 + y0 * *2) b = atan(tan) a4 = a - 2 * pi a5 = 0 a6 = pi/2 - bl =$
$[a4, a5, a6] return nlprint('\qquad\qquad ', lastthere(best_i ndividual))$

第二问代码"' 层次分析 AHP"' import numpy as np from numpy import linalg np.set$_p rintoptions(precision = 4)'''\qquad '''def arithmetic_m ean(a) : n = len(a) b = sum(a) print('b :'$
$, b) \quad normal_a = a/b print("\qquad - \qquad ") print(normal_a) average_w eight = [] for i in range(n) :$

$s = sum(normal_a[i])print(" \quad ".format(i+1), s) \quad average_weight.append(s/n)print(average_weight)pr$
$, np.array(average_weight))$

return np.array(average$_weight$)

"' 几何平均法求权重"' def geometric$_mean(a) : n = len(a)1 \qquad b = np.prod(a, 1)print(b)c =$
$pow(b, 1/n)print(c) \quad average_weight = c/sum(c)print(' \qquad :', average_weight)returnaverage_weight$

"' 特征值法求权重"' def eigenvalue(a): w, v = np.linalg.eig(a) for i in range(len(w)):
print(' 特征值', a[i], ' 特征向量', v[:, i]) index = np.argmax(w) w$_max = np.real(w[index])vector =$
$v[:, index]vector_final = np.transpose(np.real(vector))print(' \qquad :', w_max,' CR :', (w_max-$
$3)/2)normalized_weight = vector_final/sum(vector_final)print(' \qquad :', normalized_weight)returnw_ma$

"' 综合平均权重"' def average$_Weight(a) : am = arithmetic_mean(a)gm = geometric_mean(a)ev =$
$eigenvalue(a)[1]aw = np.array([am, gm, ev])print(aw)final_weight = sum(aw)/3print(final_weight)re$

a=np.array([[1,5,7], [1/5,1,5], [1/7,1/5,1]]) print(' 综合三者权重为',average$_Weight(a))$

"' 第二问优化"' import matplotlib.pyplot as plt import numpy as np from math
import cos, sin, pi, acos, atan, sqrt

Constants and parameters POP$_SIZE = 200NUM_PARAMS = 3NUM_GENERATIONS =$
$100MUTATION_RATE = 0.1CROSSOVER_PROB = 0.8$

Initialize population def initialize$_population(pop_size, num_params) : returnnp.random.rand(pop_size$

Compute end-effector position def endpoint(x): a1 = (320*x[0] - 160) * pi / 180 a2
= (165*x[1] - 60) * pi / 180 a3 = (280*x[2] - 200) * pi / 180

l1 = 300 R2$_1 = np.array([[cos(a1), 0, sin(a1), l1 * cos(a1)], [sin(a1), 0, -cos(a1), l1 *$
$sin(a1)], [0, 1, 0, 600], [0, 0, 0, 1]])$

l2 = 1200 R3$_2 = np.array([[cos(a2), -sin(a2), 0, l2*cos(a2+pi/2)], [sin(a2), cos(a2), 0, l2*$
$sin(a2 + pi/2)], [0, 0, 1, 0], [0, 0, 0, 1]])$

l3$_4 = 1500X = np.array([l3_4 * cos(pi/2 + a3), l3_4 * sin(pi/2 + a3), 0, 1])$

R = R2$_1@R3_2y = np.dot(R, X)$

return y 定义重力势能 def delta$_Ep(x) : a1 = (320 * x[0] - 160) * pi/180a2 = (165 *$
$x[1] - 60) * pi/180a3 = (280 * x[2] - 200) * pi/180l2 = 1200l1 = 300$

R2$_1 = np.array([[cos(a1), 0, sin(a1), l1*cos(a1)], [sin(a1), 0, -cos(a1), l1*sin(a1)], [0, 1, 0, 600], [0, 0$
$np.array([l2 * cos(pi/2 + a2), l2 * sin(pi/2 + a2), 0, 1])P3 = np.dot(R2_1, P3_2)z3 = P3[2] *$
$10 * *(-3)z4 = endpoint(x)[2] * 10 * *(-3)z2 = 0.6L = 0.6 + 0.3 + 1.2 + 1.5M = 5g =$
$9.8Ef = (0.3/L)*z2*M*g+(1.2/L)*M*g*(z2+z3)/2+(1.5/L)*M*g*(z4+z3)/2E0 =$
$M * g * L/2returnE0 - Ef$

Fitness function (RMSE) def RMSE(x): P = endpoint(x) target = np.array([1500, 1200, 200]) return np.sqrt(np.sum((P[:3] - target)**2)) def fitness(x): return 0.7086*RMSE(x)+0.2228*d

Tournament selection def tournament$_s election(population, fitness_s cores, tournament_s ize =$ 3) : $selected_p arents = []pop_s ize = len(population)$

for $_i nrange(pop_s ize) : tournament_i ndices = np.random.choice(pop_s ize, size = tournament_s ize, re$ $False)tournament_f itness = fitness_s cores[tournament_i ndices]selected_p arents.append(population[tou$

return np.array(selected$_p arents$)

Single-point crossover def crossover(parents, crossover$_p rob = CROSSOVER_P ROB)$: $ifnp.random.rand() < crossover_p rob : crossover_p oint = np.random.randint(1, len(parents[0]))child1$ $np.concatenate([parents[0][: crossover_p oint], parents[1][crossover_p oint :]])child2 = np.concatenate([pa$ $crossover_p oint], parents[0][crossover_p oint :]])else : child1, child2 = parents[0].copy(), parents[1].copy()$

return child1, child2

Mutation def mutate(child, mutation$_r ate = MUTATION_R ATE) : foriinrange(len(child)) :$ $ifnp.random.rand() < mutation_r ate : child[i] = np.random.rand()returnchild$

Genetic algorithm def genetic$_a lgorithm(num_g enerations, pop_s ize, num_p arams) : population =$ $initialize_p opulation(pop_s ize, num_p arams)best_f itness_h istory = []$

for generation in range(num$_g enerations) : fitness_s cores = np.array([fitness(x)forxinpopulation$ $np.min(fitness_s cores)best_i ndividual = population[np.argmin(fitness_s cores)]best_f itness_h istory.appe$

next$_p opulation = [best_i ndividual]Elitism : preservethebestindividual$

while len(next$_p opulation) < pop_s ize : parents = tournament_s election(population, fitness_s cores)c$ $crossover(parents)child1 = mutate(child1)child2 = mutate(child2)next_p opulation.extend([child1, chi$

population = np.array(next$_p opulation[: pop_s ize])$

Final best individual and fitness final$_f itness_s cores = np.array([fitness(x)forxinpopulation])best_i$ $population[np.argmin(final_f itness_s cores)]best_f itness = np.min(final_f itness_s cores)$ $plt.figure($ $(8, 6))plt.plot(range(1, len(best_f itness_h istory)+1), best_f itness_h istory, marker =' o', linestyle =$ $-', color =' b')plt.xlabel('Generation')plt.ylabel('Fitness')plt.title('FitnessChangeinGeneticAlgorith$

Run the genetic algorithm best$_i ndividual, best_f itness, best_f itness_h istory = genetic_a lgorithm(NU$ $", best_i ndividual)print("$ mm : ", $RMSE(best_i ndividual))print('$ J ', $delta_E p(best_i ndividua$

"' 求解后三个调控参数"' 输出目标点在坐标系四中的坐标, 为了求解关节 4,5,6 转角 def spacechange(x): a1=(320*x[0]-160)*pi/180 a2=(165*x[1]-60)*pi/180 a3=(280*x[2]-200)*pi/180 l1=300 R2$_1 = np.array([[cos(a1), 0, sin(a1), l1*cos(a1)], [sin(a1), 0, -cos(a1), l1*$ $sin(a1)], [0, 1, 0, 600], [0, 0, 0, 1]])l2 = 1200R3_2 = np.array([[cos(a2), -sin(a2), 0, l2*cos(a2+$

$pi/2)], [sin(a2), cos(a2), 0, l2{*}sin(a2{+}pi/2)], [0, 0, 1, 0], [0, 0, 0, 1]])l3_4 = 1500R4_3 = np.array([[0, cos(a3)$
$a3), l3_4{*}cos(pi/2{+}a3)], [0, sin(a3), sin(pi/2{+}a3), sin(pi/2{+}a3){*}l3_4], [1, 0, 0, 0], [0, 0, 0, 1]])X =$
$np.array([1500, 1200, 200, 1])R = R2_1@R3_2@R4_3 \quad A \quad R_inv = np.linalg.inv(R)y = np.dot(R_inv, X)ret$
$P = spacechange(x)x0 = P[0]y0 = P[1]z0 = P[2]cos = x0/sqrt(x0 * *2 + y0 * *2)a =$
$acos(cos)tan = z0/sqrt(x0{**}2{+}y0{**}2)b = atan(tan)a4 = a{-}2{*}pia5 = 0a6 = pi/2{-}bl =$
$[a4, a5, a6]returnlprint('\qquad\qquad', lastthere(best_individual))$

"' 求解能耗"' import scipy.integrate as spi "' 求解能耗 theta0=[0,-pi/2,0,pi,-pi/2,0]
$w_mean = [2.0, 1.5, 1.0, 2.5, 3.0, 2.0]I = [0.5, 0.3, 0.4, 0.6, 0.2, 0.4]defe_cost(x) : l = []foriinrange(0, len(x)$
$theta_f = x[i] + theta0[i]w0 = w_mean[i]I0 = I[i]t = abs(x[i]) * 180/(pi * w0)xita =$
$np.array([theta0[i], theta_f, 0, 0])Rt = np.array([[1, 0, 0, 0], [1, t, t{**}2, t{**}3], [0, 1, 0, 0], [0, 1, 2{*}$
$t, 3{*}t{**}2]])Rt_inv = np.linalg.inv(Rt)aa = np.dot(Rt_inv, xita)defomega_2(t) : return(aa[1]+$
$2 * aa[2] * t + 3 * aa[3] * t * *2) * *2time = np.linspace(0, t, 1000) \quad quad \qquad result, error =$
$spi.quad(omega_2, 0, t) \qquad\qquad r = result{*}0.5{*}I0/tl.append(r)returnlprint('\qquad', e_cost(theta))$

"'

theta0 = [0, -pi/2, 0, pi, -pi/2, 0] $w_mean = [2.0, 1.5, 1.0, 2.5, 3.0, 2.0]I = [0.5, 0.3, 0.4, 0.6, 0.2, 0.4]$

def $e_cost(theta) : l = []foriinrange(len(theta)) : ifi! = 4 : theta_f = theta[i] +$
$theta0[i]w0 = w_mean[i]I0 = I[i]t = abs(theta[i]) * 180/(pi * w0)$

xita = np.array([theta0[i], $theta_f, 0, 0])Rt = np.array([[1, 0, 0, 0], [1, t, t{**}2, t{**}3], [0, 1, 0, 0], [0, 1, 2{*}$
$t, 3 * t * *2]])$

使用 numpy.linalg.solve 求解线性方程组，代替求逆 aa = np.linalg.solve(Rt, xita)

def $omega_2(t) : return(aa[1] + 2 * aa[2] * t + 3 * aa[3] * t * *2) * *2$

调整积分函数的定义域和精度 time = np.linspace(0, t, 1000) result, error = spi.quad($omega_2, 0, t, e$
$1.0e - 6, epsrel = 1.0e - 6)$

r = result * 0.5 * I0 / t l.append(r) else: l.append(0)

return l t=lastthere($best_individual)tt = np.array(best_individual)theta = np.append(tt, t)$

print(' 六个关节能耗为：', $e_cost(theta))$ $\qquad importheapq$

class HamiltonianCycleSolver: def $_{init}(self, graph){:}self.graph{=}graphself.num_vertices{=}len(graph)self.min_cost{=}float('inf')self.$

def solve(self): 使用优先队列（最小堆）来实现 Branch and Bound 算法 priority$_queue = []initial_path = [0] \quad 0 \quad initial_cost = 0heapq.heappush(priority_queue, (initial_cost, initial_path)$

while priority$_queue : current_cost, current_path = heapq.heappop(priority_queue)$

如果当前路径已经包含了所有顶点,并且形成了一个哈密尔顿圆路径 if len(current$_path) ==$
$self.num_verticesandself.graph[current_path[-1]][current_path[0]]! = 0 : current_cost+ =$

$self.graph[current_path[-1]][current_path[0]] \qquad if current_cost < self.min_cost : self.min_cost = current_cost self.min_path = current_path[:] continue$

检查每个顶点是否可以加入到当前路径中 for vertex in range(self.num$_vertices$) : $if(len(current_path) < self.num_vertices and vertex not in current_path and self.graph[current_path[-1]][ve$ 0) :

计算新路径的代价 new$_cost = current_cost + self.graph[current_path[-1]][vertex]$

只将代价小于当前最小代价的路径加入优先队列 if new$_cost < self.min_cost : new_path = current_path + [vertex] heapq.heappush(priority_queue, (new_cost, new_path))$

return self.min$_cost, self.min_path$

示例用法 if $_{name==''main''}$: $\quad graph=[[0,16,17,25,22,1000],[16,0,11,1000,1000,24],[17,11,0,10,1000,21],[25,1000,10,0,13,15],[22,1000,1000,13,0,10$

solver = HamiltonianCycleSolver(graph) min$_cost, min_path = solver.solve()$

print(f" 最小哈密尔顿圆路径的代价为: min$_cost$")$print(f" \qquad : min_path")$

clc;clear; position$_mat = readmatrix('\ .xlsx', 'Sheet', 'Sheet1'); position_mat(isnan(position_mat)) = 0; scale = 20; maze_mat = zeros(size(position_mat) * scale); for i = 1 : size(position_mat, 1) for j = 1 : size(position_mat, 2) if position_mat(i, j) == 1 if(1 < i)(i < size(position_mat, 1))(1 < j)(j < size(position_mat, 2)) maze_mat((i - 1) * scale + 1 - ceil(scale/8) : i * scale + ceil(scale/8), ...(j - 1) * scale + 1 - ceil(scale/8) : j * scale + ceil(scale/8)) = ones...(i * scale + ceil(scale/8) - ((i - 1) * scale + 1 - ceil(scale/8)) + 1, ...j * scale + ceil(scale/8) - ((j - 1) * scale + 1 - ceil(scale/8)) + 1); else maze_mat((i - 1) * scale + 1 : i * scale, (j - 1) * scale + 1 : j * scale) = ones(scale, scale); end end end q_start = [10, 10]; q_goal = [390, 390];$

map = maze$_mat$;

k = 10000; delta$_q = 20; p = 0.3; delta = 5;$

[vertices, edges, path] = rrt(map, q$_start, q_goal, k, delta_q, p); path_smooth = smooth(map, path, vertic$ $map), []); title('RRT(Rapidly - Exploring Random Trees) - Smooth');$

hold on;

[edgesRowCount,] = size(edges);

for ii = 1 : edgesRowCount plot(vertices(ii, 1), vertices(ii, 2), 'cyan*', 'linewidth', 1); plot([vertices(edges(ii, 1), 1), vertices(edges(ii, 2), 1)], ... [vertices(edges(ii, 1), 2), vertices(edges(ii, 2), 2)], ... 'b', 'LineWidth', 1); end

plot(q$_start(1), q_start(2), 'g*', 'linewidth', 1); plot(q_goal(1), q_goal(2), 'r*', 'linewidth', 1);$

[, pathCount] = size(path);

for ii = 1 : pathCount - 1 plot([vertices(path(ii), 1), vertices(path(ii + 1), 1)], ...

[vertices(path(ii), 2), vertices(path(ii + 1), 2)], ... 'r', 'LineWidth', 1); end

[, pathCount] = size(path$_s mooth$);

for ii = 1 : pathCount - 1 plot([vertices(path$_s mooth(ii), 1), vertices(path_s mooth(ii+$
$1), 1)], ...[vertices(path_s mooth(ii), 2), vertices(path_s mooth(ii+1), 2)], ...'black',' LineWidth', 2); end

clc;clear; scale=20; position$_m at = readmatrix('$ $.xlsx',' Sheet',' Sheet2'); [row, column] =$
$find(isnan(position_m at)); position_m at(isnan(position_m at)) = 0; maze_m at = zeros(size(position_m at)*$
$scale); fori = 1 : size(position_m at, 1) forj = 1 : size(position_m at, 2) ifposition_m at(i, j) ==$
$1if(1 < i)(i < size(position_m at, 1))(1 < j)(j < size(position_m at, 2)) maze_m at((i - 1) *$
$scale + 1 - ceil(scale/8) : i * scale + ceil(scale/8), ...(j - 1) * scale + 1 - ceil(scale/8) : j *$
$scale + ceil(scale/8)) = ones...(i * scale + ceil(scale/8) - ((i - 1) * scale + 1 - ceil(scale/8)) +$
$1, ...j * scale + ceil(scale/8) - ((j - 1) * scale + 1 - ceil(scale/8)) + 1); elsemaze_m at((i - 1) *$
$scale + 1 : i * scale, (j - 1) * scale + 1 : j * scale) = ones(scale, scale); endendendendkey_p osition =$
$[row, column]; key_p osition = sortrows(key_p osition, 1); key_s pots = [(key_p osition(:, 2) -$
$1) * scale + ceil(scale/2), (key_p osition(:, 1) - 1) * scale + ceil(scale/2)]; order = [0, 1, 2, 5, 3, 4, 0]; map =$
$maze_m at;$

k = 10000; delta$_q$ = 20; $p = 0.3; delta = 5; path = ; path_s mooth = ; vertices =$
$; edges = ; fori = 1 : size(key_s pots) + 1 whiletruetryq_s tart = key_s pots(order(i) + 1, :$
$); q_g oal = key_s pots(order(i+1)+1, :); [vertices_s tep, edges_s tep, path_s tep] = rrt(map, q_s tart, q_g oal, k, delta$
$smooth(map, path_s tep, vertices_s tep, delta); pathi = path_s tep; path_s moothi = path_s mooth_s tep; verticesi$
$vertices_s tep; edgesi = edges_s tep; breakcatchcontinueendendendimshow(int32(1 - map), []); title('RRT($
$ExploringRandomTrees) - Smooth'); holdon; fori = 1 : size(key_s pots, 1) path = path_c elli; vertices =$
$vertices_c elli; edges = edges_c elli; [edgesRowCount,] = size(edges); forii = 1 : edgesRowCountplot(ve$
$1 : size(key_s pots, 1) path = path_c elli; path_s mooth = path_s mooth_c elli; vertices = vertices_c elli; edges =$
$edges_c elli; [, pathCount] = size(path);$

for ii = 1 : pathCount - 1 plot([vertices(path(ii), 1), vertices(path(ii + 1), 1)], ...
[vertices(path(ii), 2), vertices(path(ii + 1), 2)], ... 'r', 'LineWidth', 1.5); end

end for i=1:size(key$_s pots, 1) path = path_c elli; path_s mooth = path_s mooth_c elli; vertices =$
$vertices_c elli; [, pathCount] = size(path_s mooth); forii = 1 : pathCount - 1 plot([vertices(path_s mooth(ii)$
$1), 1)], ...[vertices(path_s mooth(ii), 2), vertices(path_s mooth(ii+1), 2)], ...'black',' LineWidth', 2); endendp$
$, 1), key_s pots(:, 2),' r*',' linewidth', 1.5);$

function [vertices, edges, path] = rrt(map, q$_s tart, q_g oal, k, delta_q, p) tic;$

clc;

if nargin $<$ 3 error('First 3 parameter is required: 2D map matrix, start and goal coordinates.'); elseif nargin $<$ 4 k = 10000; delta$_q = 50; p = 0.3; elseifnargin < 5delta_q = 50; p = 0.3; elseifnargin < 6p = 0.3; end$

checkParameters(map, q$_s tart, q_g oal, k, delta_q, p$);

[mapHeight, mapWidth] = size(map);

q$_s tart = int32(q_s tart); q_g oal = int32(q_g oal); vertices = q_s tart$;

edges = int32.empty(0, 2);

q$_r and = int32.empty(0, 2)$;

q$_n ear = int32.empty(0, 2)$;

q$_n ew = int32.empty(0, 2)$;

for ii = 1 : k

if rand() $<$ p q$_r and = q_g oal; elseq_r and = int32([randi(mapWidth)randi(mapHeight)]); end$

[q$_n ear, qNearIndex] = findQNear(q_r and, vertices)$;

q$_n ew = findQNew(q_n ear, q_r and, delta_q)$;

if q$_n ew(1) < 1||q_n ew(2) < 1||q_n ew(1) > mapWidth||q_n ew(2) > mapHeightcontinue; end$

if map(q$_n ew(2), q_n ew(1)) == 0$

if isEdgeQNearQNewBelongsFreeSpace(map, q$_n ear, q_n ew)$

vertices = [vertices; q$_n ew]; [qNewIndex,] = size(vertices); edges = [edges; [int32(qNewIndex), in$

if isequal(q$_n ew, q_g oal)||isQGoalOnQNearQNewEdge(q_n ear, q_n ew, q_g oal)$

if isequal(q$_n ew, q_g oal)vertices = vertices(1 : (end-1), :); vertices = [vertices; q_g oal]; end$

path = fillSolutionPath(edges, vertices);

toc;

return; end end end end

path = int32.empty(0, 2); toc;

error('RRT: solution not found :('); end

function checkParameters(map, q$_s tart, q_g oal, k, delta_q, p)$

[mapHeight, mapWidth] = size(map);

if mapWidth $<$ 1 || mapHeight $<$ 1 error('Map size error. Please provide 2D map.');

end

[x, y] = size(q$_s tart); if x = 1||y = 2error('q_s tart-startposition-shouldbe1x2matrixlike :$ $[80, 70]'); elseifq_s tart(1) < 0||q_s tart(1) > mapWidth||q_s tart(2) < 0||q_s tart(2) > mapHeighterror('q_s ta$ $startposition - shouldbeinsidethemap'); end$

[x, y] = size(q$_goal$); $if x = 1 || y = 2 error('q_goal - goalposition - shouldbe1x2matrixlike : [707, 615]'); elseif q_goal(1) < 0 || q_goal(1) > mapWidth || q_goal(2) < 0 || q_goal(2) > mapHeight error('q_goal - goalposition - shouldbeinsidethemap'); end$

if k < 1 error('k - iteration count - parameter should be positive'); end

if delta$_q$ $< 1 error('delta_q - stepsize - parametershouldbepositive'); end$

if p < 0 || p > 1 error('p - probability - parameter should be between 0 and 1'); end

end

function [q$_near$, $qNearIndex$] = $findQNear(q_rand, vertices)$

[rowCount,] = size(vertices);

if rowCount < 1 error('RRT: solution not found :('); end

euclideanDistances = double.empty(0, 1);

for ii = 1 : rowCount euclideanDistances(ii, 1) = pdist2(double(q$_rand$), $double(vertices(ii, :)), 'euclidean'); end$

minDistanceIndex = find(euclideanDistances == min(euclideanDistances));

q$_near$ = $vertices(minDistanceIndex(1), :)$;

qNearIndex = minDistanceIndex(1);

end

function [q$_new$] = $findQNew(q_near, q_rand, delta_q)$

v = double(q$_rand - q_near$);

u = v / norm(v);

q$_new$ = $int32(double(q_near) + delta_q * u)$;

end

function [isBelongsFreeSpace] = isEdgeQNearQNewBelongsFreeSpace(map, q$_near$, q$_new$)

intermediatePointCount = 10;

v = double(q$_new - q_near$);

distance = norm(v);

u = v / distance;

delta$_q$ = $distance/intermediatePointCount$;

currentCoordinate = double(q$_near$);

for ii = 1 : intermediatePointCount

currentCoordinate = currentCoordinate + (delta$_q$ $* u$);

if map(int32(currentCoordinate(2)), int32(currentCoordinate(1))) == 1 isBelongsFreeSpace = 0; return; end

end

isBelongsFreeSpace = 1;

end

function [isQGoalOnEdge] = isQGoalOnQNearQNewEdge(q$_n ear, q_n ew, q_g oal$)

v = double(q$_n ew - q_n ear$);

distance = norm(v);

u = v / distance;

distanceQNearQGoal = norm(double(q$_g oal - q_n ear$));

if distanceQNearQGoal > distance isQGoalOnEdge = 0; return; end

q$_g oal_h at = double(q_n ear) + (distanceQNearQGoal * u)$;

isQGoalOnEdge = isequal(int32(q$_g oal_h at), q_g oal$);

end

function [path] = fillSolutionPath(edges, vertices)

path = edges(end, 1);

prev = edges(end, 2);

ii = 0; [edgesCount,] = size(edges);

while prev = 1 if ii > edgesCount error('RRT: no path found :('); end prevIndex = find(edges(:, 1) == prev); prev = edges(prevIndex(1), 2); path = [path, prev]; ii = ii + 1; end

end

function rrtDraw(map, q$_s tart, q_g oal, vertices, edges, path$)

imshow(int32(1 - map), []); title('RRT (Rapidly-Exploring Random Trees)');

hold on;

[edgesRowCount,] = size(edges);

for ii = 1 : edgesRowCount plot(vertices(ii, 1), vertices(ii, 2), 'cyan*', 'linewidth', 1); plot([vertices(edges(ii, 1), 1), vertices(edges(ii, 2), 1)], ... [vertices(edges(ii, 1), 2), vertices(edges(ii, 2), 2)], ... 'b', 'LineWidth', 1); end

plot(q$_s tart(1), q_s tart(2), ' g*', ' linewidth', 1); plot(q_g oal(1), q_g oal(2), ' r*', ' linewidth', 1)$;

[, pathCount] = size(path);

for ii = 1 : pathCount - 1 plot([vertices(path(ii), 1), vertices(path(ii + 1), 1)], ... [vertices(path(ii), 2), vertices(path(ii + 1), 2)], ... 'r', 'LineWidth', 2); end

end

function [path$_s mooth$] = $smooth(map, path, vertices, delta)$

path$_s mooth = path(1); currentIndex = 1; currentSmoothIndex = numel(path);$

while currentIndex < numel(path)

while currentIndex < currentSmoothIndex

if isEdgeBelongsFreeSpace(map, vertices(path(currentSmoothIndex), :), vertices(path(currentIndex :), delta) path$_s mooth = [path_s mooth, path(currentSmoothIndex)]; currentIndex = currentSmoothIn$ $currentSmoothIndex - 1; end$

end

currentSmoothIndex = numel(path);

end

end

function [isBelongsFreeSpace] = isEdgeBelongsFreeSpace(map, startPoint, endPoint, delta)

v = double(endPoint - startPoint);

distance = norm(v);

u = v / distance;

intermediatePointCount = distance / delta;

currentCoordinate = double(startPoint);

for ii = 1 : intermediatePointCount

currentCoordinate = currentCoordinate + (delta * u);

if map(int32(currentCoordinate(2)), int32(currentCoordinate(1))) == 1 isBelongs-FreeSpace = 0; return; end

end

isBelongsFreeSpace = 1;

end

function rrtSmoothDraw(map, path$_s mooth, vertices)$

imshow(int32(1 - map), []); title('RRT (Rapidly-Exploring Random Trees) - Smooth Path');

hold on;

[, pathCount] = size(path$_s mooth$);

for ii = 1 : pathCount - 1 plot([vertices(path$_s mooth(ii), 1), vertices(path_s mooth(ii+$ $1), 1)], ...[vertices(path_s mooth(ii), 2), vertices(path_s mooth(ii{+}1), 2)], ...'r',' LineWidth', 2); end

end

第一问代码 import matplotlib.pyplot as plt import numpy as np from math import cos, sin, pi, acos, atan, sqrt import seaborn as sns sns.set() Constants and parameters POP$_S IZE = 200 NUM_P ARAMS = 3 NUM_G ENERATIONS = 100 MUTATION_R ATE =$ $0.1 CROSSOVER_P ROB = 0.8$

Initialize population def initialize$_p opulation(pop_s ize, num_p arams) : returnnp.random.rand(pop_s ize$

Compute end-effector position def endpoint(x): a1 = (320*x[0] - 160) * pi / 180 a2 = (165*x[1] - 60) * pi / 180 a3 = (280*x[2] - 200) * pi / 180

l1 = 300 R2$_1 = np.array([[cos(a1), 0, sin(a1), l1 * cos(a1)], [sin(a1), 0, -cos(a1), l1 *$ $sin(a1)], [0, 1, 0, 600], [0, 0, 0, 1]])$

l2 = 1200 R3$_2 = np.array([[cos(a2), -sin(a2), 0, l2{*}cos(a2{+}pi/2)], [sin(a2), cos(a2), 0, l2{*}$ $sin(a2 + pi/2)], [0, 0, 1, 0], [0, 0, 0, 1]])$

l3$_4 = 1500 X = np.array([l3_4 * cos(pi/2 + a3), l3_4 * sin(pi/2 + a3), 0, 1])$

R = R2$_1$@$R3_2 y = np.dot(R, X)$

return y

Fitness function (RMSE) def fitness(x): P = endpoint(x) target = np.array([1500, 1200, 200]) return np.sqrt(np.sum((P[:3] - target)**2))

Tournament selection def tournament$_s election(population, fitness_s cores, tournament_s ize =$ $3) : selected_p arents = []pop_s ize = len(population)$

for $_i nrange(pop_s ize) : tournament_i ndices = np.random.choice(pop_s ize, size = tournament_s ize, re$ $False) tournament_f itness = fitness_s cores[tournament_i ndices] selected_p arents.append(population[tou$

return np.array(selected$_p arents)$

Single-point crossover def crossover(parents, crossover$_p rob = CROSSOVER_P ROB) :$ $if np.random.rand() < crossover_p rob : crossover_p oint = np.random.randint(1, len(parents[0])) child1$ $np.concatenate([parents[0][: crossover_p oint], parents[1][crossover_p oint :]]) child2 = np.concatenate([pa$ $crossover_p oint], parents[0][crossover_p oint :]]) else : child1, child2 = parents[0].copy(), parents[1].copy()$

return child1, child2

Mutation def mutate(child, mutation$_r ate = MUTATION_R ATE) : for i in range(len(child)) :$ $if np.random.rand() < mutation_r ate : child[i] = np.random.rand() return child$

Genetic algorithm def genetic$_algorithm(num_generations, pop_size, num_params) : population = initialize_population(pop_size, num_params) best_fitness_history = []$

for generation in range(num$_generations) : fitness_scores = np.array([fitness(x) for x in population$
$np.min(fitness_scores) best_individual = population[np.argmin(fitness_scores)] best_fitness_history.appe$

next$_population = [best_individual] Elitism : preserve the best individual$

while len(next$_population) < pop_size : parents = tournament_selection(population, fitness_scores) c$
$crossover(parents) child1 = mutate(child1) child2 = mutate(child2) next_population.extend([child1, chi$

population = np.array(next$_population[: pop_size])$

Final best individual and fitness final$_fitness_scores = np.array([fitness(x) for x in population]) best_i$
$population[np.argmin(final_fitness_scores)] best_fitness = np.min(final_fitness_scores)$

创建图像 plt.figure(figsize=(8, 6))

使用 Seaborn 的线性回归图绘制数据 sns.lineplot(x=range(1, len(best$_fitness_history)+$
$1), y = best_fitness_history, marker =' o', color =' b')$

设置标签和标题 plt.xlabel('Generation') plt.ylabel('Fitness') plt.title('Fitness Change in Genetic Algorithm')

显示网格 plt.grid(True)

调整布局 plt.tight$_layout()$

显示图像 plt.show()

return best$_individual, best_fitness, best_fitness_history return best_individual, best_fitness, best_fitness$

Run the genetic algorithm best$_individual, best_fitness, best_fitness_history = genetic_algorithm(NU$
$", best_individual) print("\quad : ", best_fitness)$

"' 求解后三个调控参数"' 输出目标点在坐标系四中的坐标, 为了求解关节 4,5,6 转角 def spacechange(x): a1=(320*x[0]-160)*pi/180 a2=(165*x[1]-60)*pi/180 a3=(280*x[2]-200)*pi/180 l1=300 R2$_1 = np.array([[cos(a1), 0, sin(a1), l1*cos(a1)], [sin(a1), 0, -cos(a1), l1*$
$sin(a1)], [0, 1, 0, 600], [0, 0, 0, 1]]) l2 = 1200 R3_2 = np.array([[cos(a2), -sin(a2), 0, l2*cos(a2+$
$pi/2)], [sin(a2), cos(a2), 0, l2*sin(a2+pi/2)], [0, 0, 1, 0], [0, 0, 0, 1]]) l3_4 = 1500 R4_3 = np.array([[0, cos(a3)$
$a3), l3_4*cos(pi/2+a3)], [0, sin(a3), sin(pi/2+a3), sin(pi/2+a3)*l3_4], [1, 0, 0, 0], [0, 0, 0, 1]]) X =$
$np.array([1500, 1200, 200, 1]) R = R2_1@R3_2@R4_3 \quad A \quad R_inv = np.linalg.inv(R) y = np.dot(R_inv, X) ret$
$P = spacechange(x) x0 = P[0] y0 = P[1] z0 = P[2] cos = x0/sqrt(x0 * *2 + y0 * *2) a =$
$acos(cos) tan = z0/sqrt(x0**2+y0**2) b = atan(tan) a4 = a-2*pi a5 = 0 a6 = pi/2-b l =$
$[a4, a5, a6] return l print('\qquad\qquad', last_there(best_individual))$

第二问代码"' 层次分析 AHP"' import numpy as np from numpy import linalg

np.set$_p rintoptions(precision = 4)'''$ $'''defarithmetic_m ean(a) : n = len(a)b = sum(a)print('b :'$
$, b)$ $normal_a = a/bprint("$ $-$ $")print(normal_a)average_w eight = []foriinrange(n) :$
$s = sum(normal_a[i])print("$ $".format(i+1), s)$ $average_w eight.append(s/n)print(average_w eight)pr$
$, np.array(average_w eight))$

return np.array(average$_w eight$)

"' 几何平均法求权重"' def geometric$_m ean(a) : n = len(a)1$ $b = np.prod(a, 1)print(b)c =$
$pow(b, 1/n)print(c)$ $average_w eight = c/sum(c)print('$ $:', average_w eight)returnaverage_w eight$

"' 特征值法求权重"' def eigenvalue(a): w, v = np.linalg.eig(a) for i in range(len(w)):
print(' 特征值', a[i], ' 特征向量', v[:, i]) index = np.argmax(w) w$_m ax = np.real(w[index])vector =$
$v[:, index]vector_f inal = np.transpose(np.real(vector))print('$ $:', w_m ax,' CR :', (w_m ax-$
$3)/2)normalized_w eight = vector_f inal/sum(vector_f inal)print('$ $:', normalized_w eight)returnw_m a$

"' 综合平均权重"' def average$_W eight(a) : am = arithmetic_m ean(a)gm = geometric_m ean(a)ev =$
$eigenvalue(a)[1]aw = np.array([am, gm, ev])print(aw)final_w eight = sum(aw)/3print(final_w eight)re$

a=np.array([[1,5,7], [1/5,1,5], [1/7,1/5,1]]) print(' 综合三者权重为',average$_W eight(a)$)

"' 第二问优化"' import matplotlib.pyplot as plt import numpy as np from math
import cos, sin, pi, acos, atan, sqrt

Constants and parameters POP$_S IZE = 200NUM_P ARAMS = 3NUM_G ENERATIONS =$
$100MUTATION_R ATE = 0.1CROSSOVER_P ROB = 0.8$

Initialize population def initialize$_p opulation(pop_s ize, num_p arams) : returnnp.random.rand(pop_s ize$

Compute end-effector position def endpoint(x): a1 = (320*x[0] - 160) * pi / 180 a2
= (165*x[1] - 60) * pi / 180 a3 = (280*x[2] - 200) * pi / 180

l1 = 300 R2$_1 = np.array([[cos(a1), 0, sin(a1), l1 * cos(a1)], [sin(a1), 0, -cos(a1), l1 *$
$sin(a1)], [0, 1, 0, 600], [0, 0, 0, 1]])$

l2 = 1200 R3$_2 = np.array([[cos(a2), -sin(a2), 0, l2{*}cos(a2{+}pi/2)], [sin(a2), cos(a2), 0, l2{*}$
$sin(a2 + pi/2)], [0, 0, 1, 0], [0, 0, 0, 1]])$

l3$_4 = 1500X = np.array([l3_4 * cos(pi/2 + a3), l3_4 * sin(pi/2 + a3), 0, 1])$

R = R2$_1@R3_2y = np.dot(R, X)$

return y 定义重力势能 def delta$_E p(x) : a1 = (320 * x[0] - 160) * pi/180a2 = (165 *$
$x[1] - 60) * pi/180a3 = (280 * x[2] - 200) * pi/180l2 = 1200l1 = 300$

R2$_1 = np.array([[cos(a1), 0, sin(a1), l1{*}cos(a1)], [sin(a1), 0, -cos(a1), l1{*}sin(a1)], [0, 1, 0, 600], [0, 0$
$np.array([l2 * cos(pi/2 + a2), l2 * sin(pi/2 + a2), 0, 1])P3 = np.dot(R2_1, P3_2)z3 = P3[2] *$
$10 * *(-3)z4 = endpoint(x)[2] * 10 * *(-3)z2 = 0.6L = 0.6 + 0.3 + 1.2 + 1.5M = 5g =$

$9.8Ef = (0.3/L)*z2*M*g+(1.2/L)*M*g*(z2+z3)/2+(1.5/L)*M*g*(z4+z3)/2E0 = M*g*L/2returnE0-Ef$

Fitness function (RMSE) def RMSE(x): P = endpoint(x) target = np.array([1500, 1200, 200]) return np.sqrt(np.sum((P[:3] - target)**2)) def fitness(x): return 0.7086*RMSE(x)+0.2228*d

Tournament selection def tournament$_s election(population, fitness_s cores, tournament_s ize = 3) : selected_p arents = []pop_s ize = len(population)$

for $_i nrange(pop_s ize) : tournament_i ndices = np.random.choice(pop_s ize, size = tournament_s ize, re$ $False)tournament_f itness = fitness_s cores[tournament_i ndices]selected_p arents.append(population[tou$

return np.array(selected$_p arents)$

Single-point crossover def crossover(parents, crossover$_p rob = CROSSOVER_P ROB) :$ $ifnp.random.rand() < crossover_p rob : crossover_p oint = np.random.randint(1, len(parents[0]))child1$ $np.concatenate([parents[0][: crossover_p oint], parents[1][crossover_p oint :]])child2 = np.concatenate([pa$ $crossover_p oint], parents[0][crossover_p oint :]])else : child1, child2 = parents[0].copy(), parents[1].copy()$

return child1, child2

Mutation def mutate(child, mutation$_r ate = MUTATION_R ATE) : foriinrange(len(child)) :$ $ifnp.random.rand() < mutation_r ate : child[i] = np.random.rand()returnchild$

Genetic algorithm def genetic$_a lgorithm(num_g enerations, pop_s ize, num_p arams) : population =$ $initialize_p opulation(pop_s ize, num_p arams)best_f itness_h istory = []$

for generation in range(num$_g enerations) : fitness_s cores = np.array([fitness(x)forxinpopulation$ $np.min(fitness_s cores)best_i ndividual = population[np.argmin(fitness_s cores)]best_f itness_h istory.appe$

next$_p opulation = [best_i ndividual]Elitism : preservethebestindividual$

while len(next$_p opulation) < pop_s ize : parents = tournament_s election(population, fitness_s cores)c$ $crossover(parents)child1 = mutate(child1)child2 = mutate(child2)next_p opulation.extend([child1, chi$

population = np.array(next$_p opulation[: pop_s ize])$

Final best individual and fitness final$_f itness_s cores = np.array([fitness(x)forxinpopulation])best_i$ $population[np.argmin(final_f itness_s cores)]best_f itness = np.min(final_f itness_s cores)$ $plt.figure($ $(8, 6))plt.plot(range(1, len(best_f itness_h istory)+1), best_f itness_h istory, marker =' o', linestyle =$ $-', color =' b')plt.xlabel('Generation')plt.ylabel('Fitness')plt.title('FitnessChangeinGeneticAlgorith$

Run the genetic algorithm best$_i ndividual, best_f itness, best_f itness_h istory = genetic_a lgorithm(NU$ $", best_i ndividual)print("$ mm : ", $RMSE(best_i ndividual))print('$ J ', $delta_E p(best_i ndividua$

"' 求解后三个调控参数"' 输出目标点在坐标系四中的坐标, 为了求解关节 4,5,6 转角 def spacechange(x): a1=(320*x[0]-160)*pi/180 a2=(165*x[1]-60)*pi/180 a3=(280*x[2]-

200)*pi/180 l1=300 R2$_1 = np.array([[cos(a1), 0, sin(a1), l1*cos(a1)], [sin(a1), 0, -cos(a1), l1*$
$sin(a1)], [0, 1, 0, 600], [0, 0, 0, 1]])l2 = 1200R3_2 = np.array([[cos(a2), -sin(a2), 0, l2*cos(a2+$
$pi/2)], [sin(a2), cos(a2), 0, l2*sin(a2+pi/2)], [0, 0, 1, 0], [0, 0, 0, 1]])l3_4 = 1500R4_3 = np.array([[0, cos(a3)$
$a3), l3_4*cos(pi/2+a3)], [0, sin(a3), sin(pi/2+a3), sin(pi/2+a3)*l3_4], [1, 0, 0, 0], [0, 0, 0, 1]])X =$
$np.array([1500, 1200, 200, 1])R = R2_1@R3_2@R4_3 \;\; A \quad R_inv = np.linalg.inv(R)y = np.dot(R_inv, X)ret$
$P = spacechange(x)x0 = P[0]y0 = P[1]z0 = P[2]cos = x0/sqrt(x0**2 + y0**2)a =$
$acos(cos)tan = z0/sqrt(x0**2+y0**2)b = atan(tan)a4 = a-2*pia5 = 0a6 = pi/2-bl =$
$[a4, a5, a6]returnlprint('\qquad\qquad', lastthere(best_individual))$

"' 求解能耗"' import scipy.integrate as spi "' 求解能耗 theta0=[0,-pi/2,0,pi,-pi/2,0]
w$_m ean = [2.0, 1.5, 1.0, 2.5, 3.0, 2.0]I = [0.5, 0.3, 0.4, 0.6, 0.2, 0.4]def e_cost(x) : l = []foriinrange(0, len(x)$
$theta_f = x[i] + theta0[i]w0 = w_mean[i]I0 = I[i]t = abs(x[i]) * 180/(pi * w0)xita =$
$np.array([theta0[i], theta_f, 0, 0])Rt = np.array([[1, 0, 0, 0], [1, t, t**2, t**3], [0, 1, 0, 0], [0, 1, 2*$
$t, 3*t**2]])Rt_inv = np.linalg.inv(Rt)aa = np.dot(Rt_inv, xita)defomega_2(t) : return(aa[1]+$
$2 * aa[2] * t + 3 * aa[3] * t**2)**2time = np.linspace(0, t, 1000) \;\; quad \qquad result, error =$
$spi.quad(omega_2, 0, t) \qquad\qquad r = result*0.5*I0/tl.append(r)returnlprint('\qquad', e_cost(theta))$

"'

theta0 = [0, -pi/2, 0, pi, -pi/2, 0] w$_m ean = [2.0, 1.5, 1.0, 2.5, 3.0, 2.0]I = [0.5, 0.3, 0.4, 0.6, 0.2, 0.4]$

def e$_c ost(theta) : l = []foriinrange(len(theta)) : ifi! = 4 : theta_f = theta[i] +$
$theta0[i]w0 = w_mean[i]I0 = I[i]t = abs(theta[i]) * 180/(pi * w0)$

xita = np.array([theta0[i], theta$_f, 0, 0])Rt = np.array([[1, 0, 0, 0], [1, t, t**2, t**3], [0, 1, 0, 0], [0, 1, 2*$
$t, 3 * t **2]])$

使用 numpy.linalg.solve 求解线性方程组，代替求逆 aa = np.linalg.solve(Rt, xita)

def omega$_2(t) : return(aa[1] + 2 * aa[2] * t + 3 * aa[3] * t **2) **2$

调整积分函数的定义域和精度 time = np.linspace(0, t, 1000) result, error = spi.quad(omega$_2, 0, t, e$
$1.0e-6, epsrel = 1.0e-6)$

r = result * 0.5 * I0 / t l.append(r) else: l.append(0)

return l t=lastthere(best$_individual)tt = np.array(best_individual)theta = np.append(tt, t)$

print(' 六个关节能耗为：', e$_c ost(theta))\qquad importheapq$

class HamiltonianCycleSolver: def $_{init}(self, graph): self.graph = graphself.num_vertices = len(graph)self.min_cost = float('inf')self.$

def solve(self): 使用优先队列（最小堆）来实现 Branch and Bound 算法 prior-
ity$_queue = []initial_path = [0] \;\; 0 \;\; initial_cost = 0heapq.heappush(priority_queue, (initial_cost, initial_path)$

while priority$_queue : current_cost, current_path = heapq.heappop(priority_queue)$